



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Syntax Validator

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco (2023076790)

Soto Oquendo Cristian Gabriel (2026086510)

Tacna – Perú
2026

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Documento de Estándares de Programación

Versión 1.0

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Historia de Revisión

Historial de revisiones				
Item	Fecha	Versión	Descripción	Equipo
1	04/07/2026	1.0	Versión Original	Arocutipa / Perez

Tabla de Contenidos

Estándares de Programación	4
1. OBJETIVO	4
2. DECLARACIÓN DE VARIABLES	5
2.1 Descripción de la Variable.	5
2.2 Variables de Tipo Arreglo	5
3. Definición de datos y funciones	5
3.1 Tipo de datos	5
3.2 Declaración de variables	5
3.3 Declaración de funciones	5
3.4 Control de versiones de código fuente	5
4. Procedimientos y Funciones definidos por el Usuario.	5
5. Beneficios	5
6. Conclusiones	5

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

Estándares de Programación

1. OBJETIVO

- Asegurar la validez de los datos de entrada mediante validación manual en la capa de controladores (`validation.controller.js`, `auth.controller.js`) para los campos obligatorios (`query`, `email`, `password`), reforzada con restricciones a nivel de columna (`UNIQUE NOT NULL`, `DEFAULT`) en la base de datos PostgreSQL. El proyecto no usa Triggers.
- Centralizar el análisis léxico y sintáctico en clases reutilizables (`Lexer`, `SQLParser`, `RDPParser`, `MongoParser`, `StrictMongoParser`) para evitar la duplicidad de la gramática SQL/NoSQL entre la web principal, la Skill/API pública y el CLI (`sqlcheck`).
- Registrar timestamps de auditoría (`created_at`, `last_login`, `last_activity`, fecha) en cada usuario, administrador y evento registrado en `audit_logs`. Esto es vital para trazabilidad, depuración y para mantener un historial completo de validaciones y accesos.
- Optimizar el código (lectura por streaming con `readline`, balance de llaves/corchetes para NoSQL, procesamiento por chunks) para que la validación de archivos grandes no bloquee el event loop de Node.js.
- Estructurar el sistema por capas (`routes`, `controllers`, `services`, `validator`, `middleware`) con separación de responsabilidades, siguiendo el patrón MVC. Esto permite corregir o añadir motores SQL sin afectar al resto del sistema.
- Crear consultas SQL dedicadas (`getGeneralStats`, `getActiveUsers`, `getAuditLogs`, `getTotalUsers`) para consolidar datos clave y alimentar el dashboard administrativo.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

2. DECLARACIÓN DE VARIABLES

Se propone que la declaración de variables siga las convenciones de JavaScript (Node.js) para el código de la aplicación y las convenciones de PostgreSQL para las columnas de base de datos, ajustándose al motivo para el que se requieran. La convención definida se establece tomando en consideración principalmente lo siguiente:

- El nombre debe ser descriptivo y en camelCase para variables y funciones JavaScript, evitando abreviaturas crípticas. Ejemplos reales del proyecto: ipAddress, engineDetected, hashedPassword.

- Alcance de la variable

A medida que aumenta el tamaño del proyecto, también aumenta la utilidad de reconocer rápidamente el alcance de las variables. En este proyecto esto se logra mediante convenciones de nombre: MAYÚSCULAS_CON_GUION_BAJO para constantes de módulo, camelCase normal para variables locales y module.exports (en vez de un prefijo) para controlar qué queda privado a cada archivo.

Alcance	Prefijo	Ejemplo
Global (constante de módulo)	MAYÚSCULAS	JWT_SECRET, SQL_KEYWORDS, ALL_ENGINES
Local	camelCase	ipAddress, userExists
Interno al módulo (no exportado)	Sin prefijo; se controla con module.exports	detectCategory, stripSQLComments

Estructura	Descripción de la Variable
LONGITUD MÁX.	Sin límite estricto; se prioriza claridad sobre brevedad.
FORMATO	camelCase para variables/funciones JS, PascalCase para clases, snake_case para columnas SQL.
EJEMPLO	validateQueryAuto (función), SQLParser (clase), audit_logs (tabla)

2.1 Declaración de Variables Locales (SQL)

A. Variables de Identificación (Claves Primarias y Foráneas)

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
id	SERIAL	Clave primaria autoincremental.	users, admins, audit_logs, login_history
user_id	INTEGER (FK)	Clave foránea que referencia a users.id.	login_history, active_sessions
email / correo	VARCHAR(255) UNIQUE	Identificador único de usuarios (email) y administradores (correo).	Autenticación
jwt_token	TEXT	Token JWT emitido en el login; se elimina al cerrar sesión (logout).	active_sessions

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

B. Variables de Datos Principales (Módulo Validación)

Utilizadas para registrar y procesar cada consulta SQL/NoSQL enviada a validar por la web, la Skill/API o el CLI.

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
query / code	string	Consulta SQL o comando MongoDB enviado a validar.	Validación
type / engine	string	Motor solicitado: sql, nosql, auto, mysql, postgresql, etc.	Validación
dialect / engineDetected	string	Motor detectado automáticamente por detectEngine().	Validación
valid	boolean	Resultado de validación: true si no hay errores de sintaxis.	Validación
errors	array de objetos	Lista de errores con line, column, message, suggestion.	Validación

C. Variables de Control de Flujo, Lógica y Trazabilidad

Utilizadas para registrar y actualizar la información de sesiones, auditoría y roles.

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
role / rol	string	Rol del usuario o administrador: user / admin / superadmin.	users, admins
is_online	boolean	Indica si el usuario tiene actividad reciente (< 5 minutos).	users
usuario, accion, detalles, ip	string / text	Columnas de auditoría: quién, qué acción, detalle e IP de origen.	audit_logs
created_at / last_login / last_activity	timestamp with time zone	Marcas de tiempo de auditoría de sesiones y validaciones.	Auditoría

D. Variables de Paginación y Filtrado (Para Reportes)

Utilizadas en las consultas del panel administrativo para filtrar el historial de auditoría y ordenar los resultados.

Variable	Tipo de Dato (Ejemplo)	Descripción	Módulo
filter	str	Filtra audit_logs por categoría: ALL / ERRORS / EVENTS / ACCESS / VALIDATIONS / ADMIN.	Auditoría
LIMIT 200	int	Límite de filas devueltas en getAuditLogs para proteger el rendimiento.	Auditoría
ORDER BY fecha DESC	str	Orden de los resultados en el historial de auditoría.	Historial / Reportes

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

3. Definición de datos y funciones

Se deben utilizar los siguientes tipos de datos (Node.js / PostgreSQL) para asegurar la consistencia entre la API y la base de datos.

3.1 Tipo de datos

Uso	Tipo de Dato Estándar (Postgres/Node.js)	Razón de la Elección	Ejemplos de Campos
Identificadores	SERIAL (Postgres) / number (JS)	Autoincremental simple, adecuado para el volumen de usuarios y logs del proyecto.	id, user_id
Valores booleanos	BOOLEAN (Postgres) / boolean (JS)	Tipo nativo para estados binarios.	is_online, activo, valid
Fechas y tiempos	TIMESTAMP WITH TIME ZONE (Postgres)	Conserva la zona horaria; vital para la trazabilidad de sesiones y validaciones.	created_at, last_login, fecha
Texto (sin límite fijo)	TEXT (Postgres) / string (JS)	Postgres no penaliza 'text' frente a varchar(N); se usa para contenido variable.	detalles, query, code
Enumerados	string con lista fija en el controlador	No hay CHECK constraints en la base de datos; la validación de valores permitidos se hace en el controlador/servicio.	status, role, rol, accion, engine
Númericos	SERIAL / INTEGER (Postgres), number (JS)	INTEGER para IDs y contadores; no se manejan montos ni tamaños de archivo grandes.	id, line, column
JSON (en tránsito HTTP, no persistido)	string / JSON (Node.js)	Las consultas SQL/NoSQL se reciben y devuelven como texto/JSON plano, no se persisten en la base de datos.	code, errors, suggestions

3.2 Declaración de Procedimientos Almacenados

Los módulos de servicio y controlador Node.js (validation.service.js, auth.controller.js, admin.controller.js) son la principal herramienta para encapsular el flujo de trabajo del proyecto, en ausencia de procedimientos almacenados en base de datos.

Regla	Descripción	Ejemplo (Proyecto)
Nomenclatura	Las funciones de servicio/controlador usan camelCase y la convención verbo+sustantivo.	validateQueryAuto, generateToken, createAdmin
Encapsulación	Cada acción se encapsula en un módulo exportado con module.exports.	validation.service.js, auth.controller.js, admin.controller.js
Propósito	Cada función debe ejecutar una operación completa y devolver un resultado consistente (JSON).	validateQueryAuto() retorna { valid, errors, dialect, ... }
Manejo de Errores	Se usan try/catch y next(err) hacia errorHandler.middleware.js en vez de TRY...CATCH SQL.	validateQuery() delega errores a next(err)

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

3.3 Declaración de Funciones Definidas por el Usuario

Las funciones puras (sin efectos secundarios) se utilizan para encapsular cálculos o lógica reutilizable que no modifica datos externos.

Regla	Descripción	Ejemplo (Proyecto)
Prefijo	Ninguno obligatorio; nombres descriptivos en camelCase. No hay convención de guion bajo inicial (JS controla visibilidad con module.exports).	stripSQLComments, findSimilarKeyword, levenshtein
Propósito	Se usan para cálculo o validación que no modifica datos ni estado.	detectCategory(query, type), positionToLineColumn(text, index)
Validación	Se usan para verificar condiciones antes de una operación.	preValidateSQLStructure() valida la estructura antes de tokenizar

Funciones de Tabla

Regla	Descripción	Ejemplo (Proyecto)
Prefijo	Ninguno obligatorio; se nombran según los datos que procesan/listan.	findCreateTableSections, splitTopLevelCreateItems
Propósito	Analizan una colección de datos y devuelven una lista de resultados.	collectStrictSQLIssues(query) retorna una lista de errores/advertencias

3.4 Declaración de Triggers

El proyecto no usa Triggers: la integridad de datos se refuerza con validación manual en los controladores (verificar que el email no exista antes de insertar, verificar que quede al menos un superadmin activo antes de desactivar) y la auditoría se aplica a nivel de aplicación Node.js (registro explícito en audit_logs tras cada acción sensible, con timestamps automáticos DEFAULT CURRENT_TIMESTAMP).

Regla	Descripción	Ejemplo (Proyecto)
Validación manual (controlador)	Reemplaza la integridad que un Trigger validaría en tiempo de ejecución; se implementa como verificación explícita en JavaScript antes de cada INSERT/UPDATE.	register() verifica que el email no exista; deactivateAdmin() exige que quede al menos un superadmin activo
Validación previa	Equivalente a BEFORE INSERT: se valida antes de insertar.	createAdmin() verifica que el correo no exista antes de insertar
Post-procesamiento	Equivalente a AFTER INSERT/UPDATE: pasos adicionales tras una operación exitosa.	login() actualiza last_login / last_activity / is_online e inserta en login_history tras autenticar
Auditoría	Se registran timestamps (DEFAULT CURRENT_TIMESTAMP) en cada inserción relevante.	created_at, last_login, fecha en users, admins, audit_logs

3.5 Control de versiones de código fuente

Cada modificación relevante se registra mediante un commit de Git siguiendo esta convención:

Título	Descripción
Formato	Control de versiones con Git. Commits descriptivos y versionado semántico del paquete (package.json, v1.1.0) para los releases del CLI (sqlcheck) publicados en releases/cli.
Descripción	El repositorio se aloja en GitHub, con GitHub Actions (quality.yml, deploy-aws.yml, deploy-azure.yml) ejecutando tests (Jest, Cucumber, Stryker) y despliegues en cada push/PR.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

4. Procedimientos y Funciones definidos por el Usuario.

El estándar dicta que el flujo de trabajo completo del Validador de Sintaxis SQL/NoSQL debe estar encapsulado en módulos de servicio y controlador Node.js para garantizar la integridad, el rendimiento y la reutilización del código, ya que no existen procedimientos ni funciones a nivel de base de datos.

4.1 Procedimientos Almacenados (SPs) - Flujo Transaccional

Los métodos de servicio y controlador manejan las operaciones de validación, autenticación y auditoría. Al no existir transacciones SQL explícitas, cada método debe validar antes de llamar a PostgreSQL y propagar el error mediante `next(err)` o una respuesta HTTP de error si algo falla.

Módulo/Acción	Método (equivalente a SP)	Parámetros de Entrada Clave	Propósito Principal
Validación de consulta	<code>validation.service.validateQueryAuto</code>	<code>query</code> , <code>requestedType</code>	Detecta la categoría (SQL/NoSQL) y delega al parser correspondiente.
Validación SQL	<code>SQLParser.parse</code>	<code>query</code> , <code>startLine</code>	Tokeniza y valida sintaxis SQL multimotor, retorna errores con línea/columna.
Validación NoSQL	<code>MongoParser.parse</code> / <code>StrictMongoParser.parse</code>	<code>query</code> , <code>startLine</code>	Valida sintaxis de comandos MongoDB (<code>find</code> , <code>insertOne</code> , <code>aggregate</code> , etc.).
Validación de archivo grande	<code>validation.controller.validateFile</code>	<code>req.file</code> , <code>type</code>	Procesa un archivo SQL/NoSQL por streaming (SSE) y reporta progreso y errores.
Detección de motor	<code>detector.detectEngine</code>	<code>tokens</code> , <code>queryText</code>	Calcula el motor SQL más probable (MySQL, PostgreSQL, Oracle, etc.) y su compatibilidad.
Registro / Login	<code>auth.controller.register</code> / <code>login</code> / <code>adminLogin</code>	<code>name</code> , <code>email</code> , <code>password</code>	Crea usuarios, valida credenciales, genera JWT y registra la sesión.
Auditoría de acciones	<code>admin.controller.createAdmin</code> / <code>deactivateUser</code> / <code>deleteAdmin</code>	<code>id</code> , <code>req.user</code>	Ejecuta la acción administrativa y registra el evento en <code>audit_logs</code> .

4.2 Funciones Definidas por el Usuario (UDFs) - Lógica y Consultas

Las funciones puras se utilizan para cálculos y validaciones (detección de categoría, similaridad de palabras clave, ubicación de errores) que no implican modificación de datos.

4.2.1 Funciones Escalares

Módulo/Acción	Función Requerida	Parámetros de Entrada Clave	Valor de Retorno	Propósito Principal
Autenticación	<code>auth.controller.generateToken</code>	<code>userId</code> , <code>email</code> , <code>role</code>	string (JWT)	Genera el token de sesión firmado con <code>JWT_SECRET</code> .
Detección de categoría	<code>validation.service.detectCategory</code>	<code>query</code> , <code>requestedType</code>	string ('sql'/'nosql')	Determina si una consulta es SQL o NoSQL antes de parsear.
Detección de motor	<code>detector.detectEngine</code>	<code>tokens</code> , <code>queryText</code>	object { <code>engine</code> , <code>confidence</code> }	Calcula el motor SQL más probable con score de confianza.
Similaridad de palabras	<code>sql/parser.findSimilarKeyword</code> / <code>levenshtein</code>	<code>word</code>	string / number	Sugiere la palabra clave correcta ante un typo (ej. FORM → FROM).
Ubicación de errores	<code>sql/parser.positionToLineColumn</code>	<code>text</code> , <code>index</code>	{ <code>line</code> , <code>column</code> }	Traduce un índice de carácter a línea/columna para el reporte de errores.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

4.2.2 Funciones de Tabla

Módulo/Acción	Función Requerida	Parámetros de Entrada Clave	Propósito Principal
Estructura CREATE TABLE	sql/parser.findCreateTableSections	query	Devuelve las secciones de columnas/constraints de sentencias CREATE TABLE.
División de sentencias	sql/parser.splitMySQLDelimiterScript	query, startLine	Divide un script en sentencias respetando DELIMITER (MySQL).
Recolección de errores	sql/parser.collectStrictSQLIssues	query, startLine	Devuelve la lista completa de errores/advertencias detectados en modo estricto.
Tokenización	lexer.Lexer.tokenize	input	Devuelve la lista de tokens tipados (KEYWORD, IDENTIFIER, etc.) del texto SQL.

5. Beneficios

- La validación manual de campos obligatorios (query, email, password) antes de cada operación asegura que las reglas de negocio (motor válido, consulta no vacía) se cumplan antes de tocar el parser o la base de datos. Esto es la máxima garantía contra resultados inconsistentes.
- El uso de clases reutilizables (Lexer, SQLParser, MongoParser) evita que se repita la gramática SQL/NoSQL entre la web principal, la Skill/API y el CLI, reduciendo la probabilidad de que un motor quede desincronizado respecto a otro.
- La modularidad por capas (routes, controllers, services, validator) y la nomenclatura estándar (camelCase en JS, snake_case en columnas SQL) hacen que cualquier desarrollador pueda entender, modificar o añadir un nuevo motor SQL con un impacto mínimo en el código existente.
- Gracias a los timestamps de auditoría (created_at, last_login, last_activity, fecha) registrados en usuarios, sesiones y audit_logs, se puede identificar rápidamente cuándo ocurrió un evento y su contexto, acelerando la depuración.
- El uso de consultas SQL optimizadas (getGeneralStats, getActiveUsers, getAuditLogs, límite de 200 filas) para reportes críticos asegura que el panel administrativo cumpla con los requerimientos no funcionales de rendimiento.
- El registro de cada acción sensible en audit_logs (usuario, accion, detalles, ip) proporciona evidencia auditable de qué se hizo, cuándo y por quién, cumpliendo con los requisitos de seguridad y control interno del curso de Calidad y Pruebas de Software.
- La definición de consultas dedicadas para el panel (getGeneralStats, getAuditLogs, getTotalUsers) permite extraer información analítica de manera eficiente para la toma de decisiones administrativas.

6. Conclusiones

- La arquitectura del Validador de Sintaxis SQL/NoSQL establece una estructura MVC clara y coherente que facilita la segregación de responsabilidades entre rutas (routes), controladores (controllers), servicios (services) y el motor de validación (validator).
- La aplicación de validaciones explícitas en los controladores y la tokenización estricta del Lexer/Parser garantiza un mecanismo robusto de detección de errores sintácticos, asegurando el cumplimiento de reglas críticas de negocio (motor detectado, consulta no vacía) y la trazabilidad de las validaciones y accesos registrados en audit_logs.

Proyecto Sistema de gestión de eventos	Versión: 1.0
Documento de Estándares de Programación	

- Se ha programado con éxito el flujo de trabajo completo de validación multimotor (SQL ANSI, MySQL, MariaDB, PostgreSQL, SQL Server, Oracle, SQLite y MongoDB) a través de clases de servicio dedicadas, lo que automatiza el proceso y reduce el trabajo manual de revisión de sintaxis.
- El uso de un lexer/parser propio y de consultas SQL optimizadas para el panel administrativo asegura que el sistema pueda manejar de manera eficiente el volumen de validaciones y accesos concurrentes, cumpliendo con los requerimientos no funcionales de rendimiento.
- El uso de nombres descriptivos y reglas sistemáticas de nomenclatura (camelCase en JavaScript, snake_case en columnas PostgreSQL) ha contribuido a una mejor identificación de variables y componentes, lo que reduce la posibilidad de errores y acelera los tiempos de mantenimiento.
- La definición de consultas dedicadas para reportes y auditoría (getGeneralStats, getAuditLogs, getTotalUsers) permite una extracción de datos eficiente, facilitando el análisis del uso del sistema y el cumplimiento de los objetivos de calidad del proyecto.